VNR VJIET

Name of the Laboratory:

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

1. Program to implement Priority scheduling algorithm in preemptive mode.

Aim: : To implement Priority scheduling algorithm in preemptive mode given and finding average Waiting time and average Turn around time.

Algorithm:

1. Start.
2. Read the number of processes n.
3. Read the Arrival Time (AT), Burst Time (BT), and Priority (PR) of each process.
4. Initialize:
    - Current Time t = 0
    - Completed Processes count = 0
5. Repeat until all processes are completed:
    - Find all processes whose AT ≤ t and are not completed.
    - Select the process with the **highest priority** (smallest priority number).
    - If no process is available, increment the current time by 1.
    - Otherwise:
        - Execute the selected process completely.
        - Calculate:
            - Completion Time (CT) = Current Time + Burst Time
            - Turnaround Time (TAT) = CT − AT
            - Waiting Time (WT) = TAT − BT
        - Mark the process as completed.
        - Update the current time to the Completion Time.
6. Calculate:
    - Average Waiting Time = ΣWT / n
    - Average Turnaround Time = ΣTAT / n
7. Display Process ID, Arrival Time, Burst Time, Priority, Completion Time, Turnaround Time, and Waiting Time.
8. Stop.

Problem:

Calculate Average waiting time , Average turn around time using Priority Scheduling Algorithm in preemptive mode for the following processes given below.

Process	AT	BT	Priority
P1	1	4	6
P2	2	4	7
P3	3	6	7

Program:

```
#include <stdio.h>
 int main()
{
   int n, i, time = 0, completed = 0;
   int at[20], bt[20], rt[20], pr[20];
   int ct[20], tat[20], wt[20];
   int index, minPriority;
   float avgWT = 0, avgTAT = 0;

   printf("Enter number of processes: ");
   scanf("%d", &n);

   for(i = 0; i < n; i++)
   {
      printf("Enter Arrival Time, Burst Time and Priority of P%d: ", i + 1);
      scanf("%d%d%d", &at[i], &bt[i], &pr[i]);
      rt[i] = bt[i];
   }

   while(completed < n)
   {
      index = -1;
      minPriority = 9999;

      for(i = 0; i < n; i++)
      {
         if(at[i] <= time && rt[i] > 0)
```

```
            {

          if(pr[i] < minPriority)
                {
                   minPriority = pr[i];
                   index = i;
                }
             }
            }

            if(index == -1)
            {
               time++;
            }
            else
            {
               rt[index]--;
               time++;

               if(rt[index] == 0)
               {
                  completed++;
                  ct[index] = time;
                  tat[index] = ct[index] - at[index];
                  wt[index] = tat[index] - bt[index];
avgTAT += tat[index];
                  avgWT += wt[index];
               }
            }
         }
```

```
    printf("Process\tAT\tBT\tPR\tCT\tTAT\tWT\n");
    printf("-----------------------------------------------------------------\n");
    for(i = 0; i < n; i++)
    {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
               i + 1, at[i], bt[i], pr[i],
               ct[i], tat[i], wt[i]);
    }
    printf("-----------------------------------------------------------------\n");
    printf("Average Waiting Time = %.2f\n", avgWT / n);
    printf("Average Turnaround Time = %.2f\n", avgTAT / n);

    return 0;
}
```

Output:

Testcase 1:

```
"C:\Users\harsh\OneDrive\Documents\OS PROGRAMS\PRIOR_PREEMPTIVE.exe"
Enter number of processes: 3
Enter Arrival Time, Burst Time and Priority of P1: 1 4 6
Enter Arrival Time, Burst Time and Priority of P2: 2 4 7
Enter Arrival Time, Burst Time and Priority of P3: 3 6 7

-----------------------------------------------------------------
Process AT      BT      PR      CT      TAT     WT
-----------------------------------------------------------------
P1      1       4       6       5       4       0
P2      2       4       7       9       7       3
P3      3       6       7       15      12      6
-----------------------------------------------------------------
Average Waiting Time = 3.00
Average Turnaround Time = 7.67

Process returned 0 (0x0)   execution time : 24.522 s
Press any key to continue.
```

Testcase 2:

```
"C:\Users\harsh\OneDrive\Documents\OS PROGRAMS\PRIOR_PREEMPTIVE.exe"
Enter number of processes: 3
Enter Arrival Time, Burst Time and Priority of P1: 0 2 1
Enter Arrival Time, Burst Time and Priority of P2: 2 3 4
Enter Arrival Time, Burst Time and Priority of P3: 4 5 6

-----------------------------------------------------------------
Process AT      BT      PR      CT      TAT     WT
-----------------------------------------------------------------
P1      0       2       1       2       2       0
P2      2       3       4       5       3       0
P3      4       5       6       10      6       1
-----------------------------------------------------------------
Average Waiting Time = 0.33
Average Turnaround Time = 3.67

Process returned 0 (0x0)   execution time : 18.414 s
Press any key to continue.
```

Testcase 3:

```
"C:\Users\harsh\OneDrive\Documents\OS PROGRAMS\PRIOR_PREEMPTIVE.exe"
Enter number of processes: 4
Enter Arrival Time, Burst Time and Priority of P1: 1 2 3
Enter Arrival Time, Burst Time and Priority of P2: 2 3 4
Enter Arrival Time, Burst Time and Priority of P3: 3 4 5
Enter Arrival Time, Burst Time and Priority of P4: 4 5 6

-----------------------------------------------------------------
Process AT      BT      PR      CT      TAT     WT
-----------------------------------------------------------------
P1      1       2       3       3       2       0
P2      2       3       4       6       4       1
P3      3       4       5       10      7       3
P4      4       5       6       15      11      6
-----------------------------------------------------------------
Average Waiting Time = 2.50
Average Turnaround Time = 6.00

Process returned 0 (0x0)   execution time : 18.062 s
Press any key to continue.
```

2.Program to implement Priority scheduling algorithm in non-preemptive mode.

Aim: : To implement Priority scheduling algorithm in non-preemptive mode given and finding average Waiting time and average Turn around time.

Algorithm:

1. Start.
2. Read the number of processes n.
3. Read the Arrival Time (AT), Burst Time (BT), and Priority (PR) of each process.
4. Set the current time t = 0 and completed processes count = 0.
5. While all processes are not completed:
    - Find all processes that have arrived ($AT \leq t$) and are not completed.
    - Select the process with the highest priority (smallest priority number).
    - If no process has arrived, increment the current time by 1.
    - Otherwise:
        - Execute the selected process until completion.
        - Calculate:
            - Completion Time (CT) = Current Time + BT
            - Turnaround Time (TAT) = CT − AT
            - Waiting Time (WT) = TAT − BT
        - Mark the process as completed.
        - Update the current time to the completion time.
6. Repeat Step 5 until all processes are completed.
7. Calculate:
    - Average Waiting Time = ΣWT / n
    - Average Turnaround Time = ΣTAT / n
8. Display Process ID, AT, BT, Priority, CT, TAT, and WT.
9. Stop.

Problem:

Calculate Average waiting time , Average turn around time using Priority Scheduling Algorithm in non-preemptive mode for the following processes given below.

Problem	AT	BT	Priority
P1	2	4	2
P2	4	6	1
P3	7	9	4
P4	5	5	3

VNR VJIET

Name of the Laboratory:

Name of the Experiment:_____

Experiment No:_____ **Date:**_____

Program:

```
#include <stdio.h>
int main()
 {
    int n, i, time = 0, completed = 0;
    int at[20], bt[20], pr[20];
    int ct[20], tat[20], wt[20];
    int visited[20] = {0};
    int highest, minPr;
    float avgWT = 0, avgTAT = 0;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    for(i = 0; i < n; i++)
    {
       printf("Enter Arrival Time, Burst Time and Priority of P%d: ", i + 1);
       scanf("%d%d%d", &at[i], &bt[i], &pr[i]);
    }
    while(completed < n)
    {
       highest = -1;
       minPr = 9999;
       for(i = 0; i < n; i++)
       {
          if(at[i] <= time && visited[i] == 0)
          {
             if(pr[i] < minPr)
             {
                minPr = pr[i];
                highest = i;
```

```
            }
            else if(pr[i] == minPr)
            {

                if(highest == -1 || at[i] < at[highest])
                {
                    highest = i;
                }
            }
        }
    }


    if(highest == -1)
    {
        time++;
        continue;
    }

    time += bt[highest];
    ct[highest] = time;
    tat[highest] = ct[highest] - at[highest];
    wt[highest] = tat[highest] - bt[highest];

    avgWT += wt[highest];
    avgTAT += tat[highest];

    visited[highest] = 1;
    completed++;
}

printf("\n----------------------------------------------------------\n");
```

```
    printf("Process\tAT\tBT\tPR\tCT\tTAT\tWT\n");

    printf("-----------------------------------------------------------\n");

    for(i = 0; i < n; i++)
    {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
               i + 1, at[i], bt[i], pr[i],
               ct[i], tat[i], wt[i]);
    }

    printf("-----------------------------------------------------------\n");
    printf("Average Waiting Time = %.2f\n", avgWT / n);
    printf("Average Turnaround Time = %.2f\n", avgTAT / n);

    return 0;
}
```

OUTPUT:

Test Case1:

```
"C:\Users\harsh\OneDrive\Documents\OS PROGRAMS\PRIOR_NONPREEMPTIVE.exe"
Enter number of processes: 4
Enter Arrival Time, Burst Time and Priority of P1: 2 4 2
Enter Arrival Time, Burst Time and Priority of P2: 4 6 1
Enter Arrival Time, Burst Time and Priority of P3: 7 9 4
Enter Arrival Time, Burst Time and Priority of P4: 5 5 3

-----------------------------------------------------------
Process AT      BT      PR      CT      TAT     WT
-----------------------------------------------------------
P1      2       4       2       6       4       0
P2      4       6       1       12      8       2
P3      7       9       4       26      19      10
P4      5       5       3       17      12      7
-----------------------------------------------------------
Average Waiting Time = 4.75
Average Turnaround Time = 10.75

Process returned 0 (0x0)   execution time : 50.298 s
Press any key to continue.
```

Testcase 2:

```
"C:\Users\harsh\OneDrive\Documents\OS PROGRAMS\PRIOR_NONPREEMPTIVE.exe"
Enter number of processes: 3
Enter Arrival Time, Burst Time and Priority of P1: 2 3 1
Enter Arrival Time, Burst Time and Priority of P2: 4 4 3
Enter Arrival Time, Burst Time and Priority of P3: 3 2 2

------------------------------------------------------------
Process AT      BT      PR      CT      TAT     WT
------------------------------------------------------------
P1      2       3       1       5       3       0
P2      4       4       3       11      7       3
P3      3       2       2       7       4       2
------------------------------------------------------------
Average Waiting Time = 1.67
Average Turnaround Time = 4.67

Process returned 0 (0x0)   execution time : 20.456 s
Press any key to continue.
```

Testcase 3:

```
"C:\Users\harsh\OneDrive\Documents\OS PROGRAMS\PRIOR_NONPREEMPTIVE.exe"
Enter number of processes: 5
Enter Arrival Time, Burst Time and Priority of P1: 1  2 3
Enter Arrival Time, Burst Time and Priority of P2: 2 3 4
Enter Arrival Time, Burst Time and Priority of P3: 3 4 5
Enter Arrival Time, Burst Time and Priority of P4: 4 5 6
Enter Arrival Time, Burst Time and Priority of P5: 5 6 7

------------------------------------------------------------
Process AT      BT      PR      CT      TAT     WT
------------------------------------------------------------
P1      1       2       3       3       2       0
P2      2       3       4       6       4       1
P3      3       4       5       10      7       3
P4      4       5       6       15      11      6
P5      5       6       7       21      16      10
------------------------------------------------------------
Average Waiting Time = 4.00
Average Turnaround Time = 8.00

Process returned 0 (0x0)   execution time : 19.665 s
Press any key to continue.
```

3. Program to implement Round Robin with Quantum values 2 and 10 scheduling algorithm in non-preemptive mode.

Aim: To implement Round Robinscheduling algorithm with Quantum values 2 and 10 given and finding average Waiting time and average Turn around time.

Algorithm:

1. Start.
2. Read the number of processes n.
3. Read the Burst Time (BT) for each process.
4. Read the Time Quantum (TQ).
5. Initialize:
    - Remaining Time (RT) = BT
    - Current Time = 0
6. Repeat until all processes are completed:
    - Traverse all processes.
    - If a process has remaining burst time:
        - If RT > TQ
            - Execute it for TQ units.
            - Reduce remaining time by TQ.
            - Increase current time by TQ.
        - Otherwise
            - Execute it for the remaining time.
            - Increase current time.
            - Calculate:
                - CT = Current Time
                - TAT = CT
                - WT = TAT − BT
7. Calculate:
    - Average Waiting Time
    - Average Turnaround Time
8. Display the results.

9. Stop.

Problem:

Process	AT	BT
P1	1	5
P2	2	6
P3	3	6

Program:

```
#include <stdio.h>

int main()
{
   int n, tq;
   int at[20], bt[20], rt[20];
   int ct[20], tat[20], wt[20];
   int completed = 0, time = 0;
   int i, found;

   printf("Enter number of processes: ");
   scanf("%d", &n);

   for(i = 0; i < n; i++)
   {
      printf("Enter Arrival Time and Burst Time of P%d: ", i + 1);
      scanf("%d%d", &at[i], &bt[i]);
      rt[i] = bt[i];
   }

   printf("Enter Time Quantum: ");
   scanf("%d", &tq);

   while(completed < n)
   {
      found = 0;

      for(i = 0; i < n; i++)
```

```
    {
        if(at[i] <= time && rt[i] > 0)
        {
            found = 1;

            if(rt[i] > tq)
            {
                time += tq;
                rt[i] -= tq;
            }
            else
            {
                time += rt[i];
                rt[i] = 0;
                ct[i] = time;
                completed++;
            }
        }
    }

    if(found == 0)
        time++;
}

float avgWT = 0, avgTAT = 0;

printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");

for(i = 0; i < n; i++)
{
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];
```

```
    avgWT += wt[i];
    avgTAT += tat[i];

    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        i + 1, at[i], bt[i], ct[i], tat[i], wt[i]);
  }

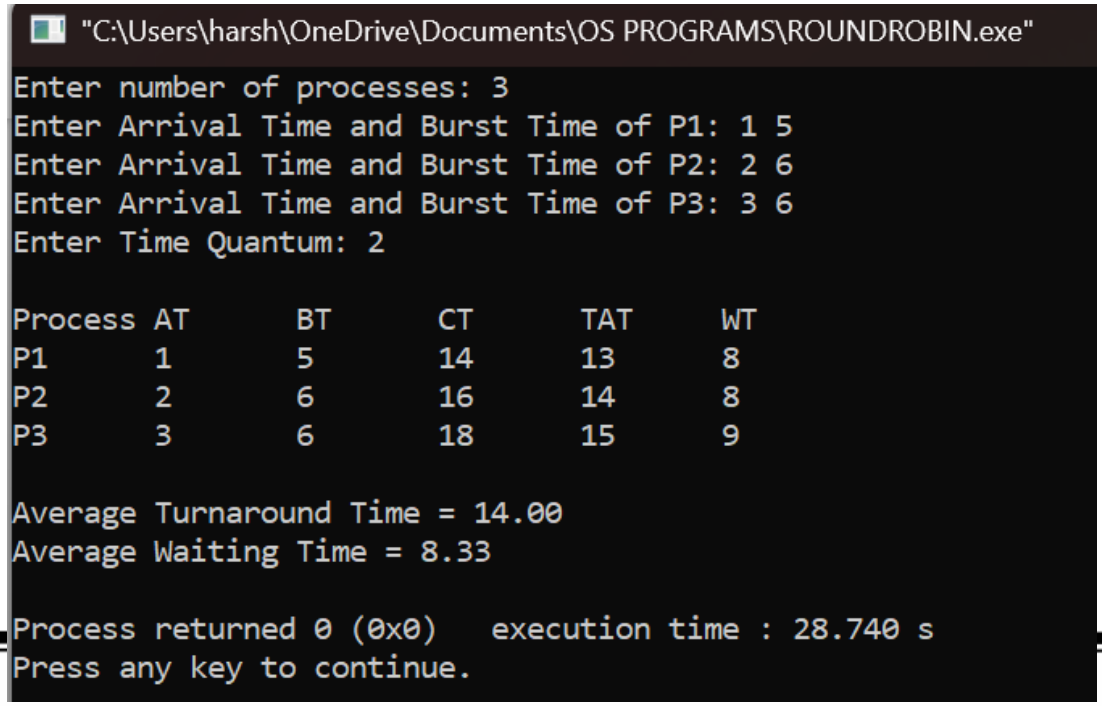
  avgWT /= n;
  avgTAT /= n;

  printf("\nAverage Turnaround Time = %.2f", avgTAT);
  printf("\nAverage Waiting Time = %.2f\n", avgWT);

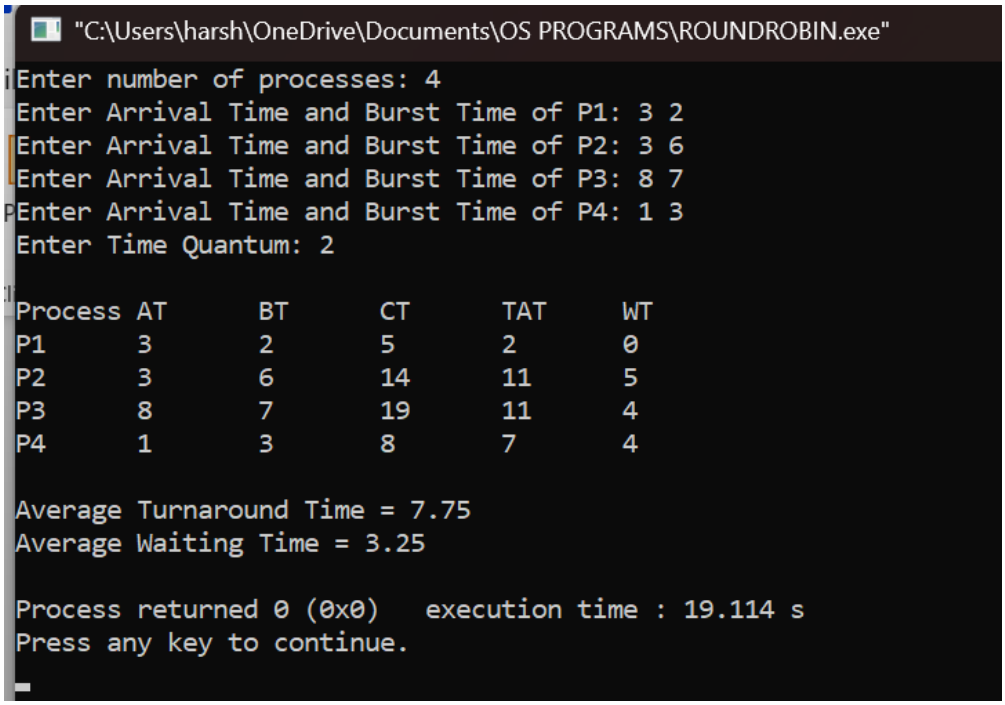
  return 0;
}
```

Output:

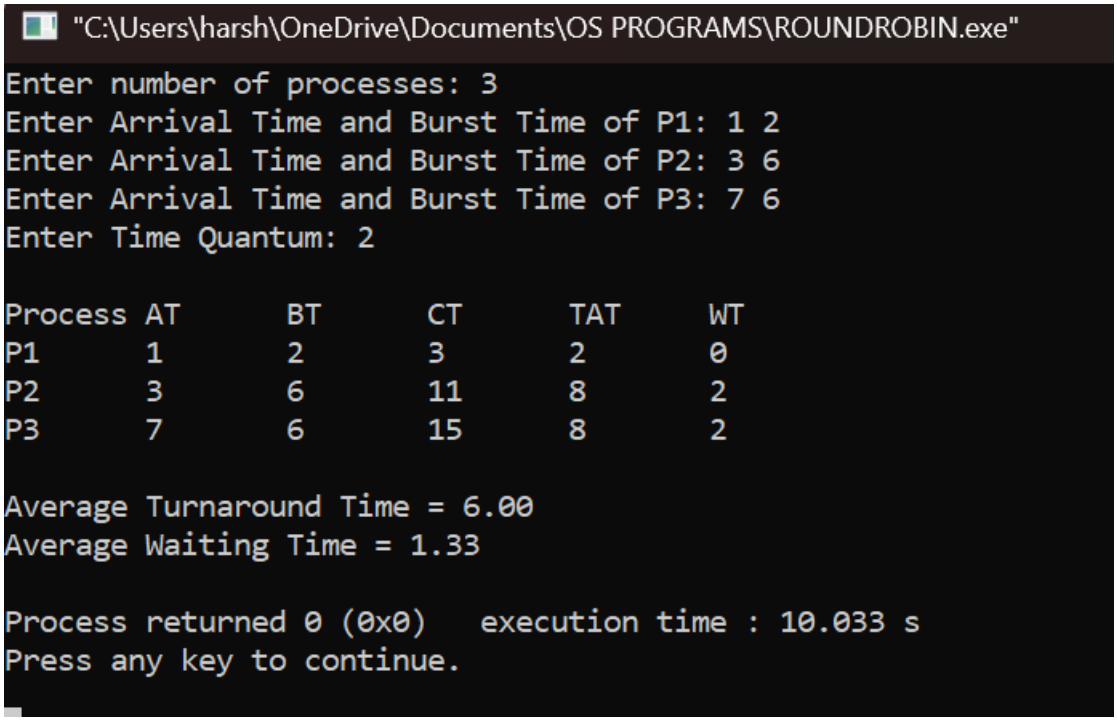
Testcase 1:



Testcase 2:



Testcase 3:


"C:\Users\harsh\OneDrive\Documents\OS PROGRAMS\ROUNDROBIN.exe"
Enter number of processes: 3
Enter Arrival Time and Burst Time of P1: 1 2
Enter Arrival Time and Burst Time of P2: 3 6
Enter Arrival Time and Burst Time of P3: 7 6
Enter Time Quantum: 2
Process AT BT CT TAT WT
P1 1 2 3 2 0
P2 3 6 11 8 2
P3 7 6 15 8 2
Average Turnaround Time = 6.00
Average Waiting Time = 1.33
Process returned 0 (0x0) execution time : 10.033 s
Press any key to continue.

With quantum 10:

```
"C:\Users\harsh\OneDrive\Documents\OS PROGRAMS\ROUNDROBIN.exe"
Enter number of processes: 4
Enter Arrival Time and Burst Time of P1: 1 2
Enter Arrival Time and Burst Time of P2: 3 4
Enter Arrival Time and Burst Time of P3: 4 5
Enter Arrival Time and Burst Time of P4: 5 6
Enter Time Quantum: 10

Process AT      BT      CT      TAT     WT
P1      1       2       3       2       0
P2      3       4       7       4       0
P3      4       5       12      8       3
P4      5       6       18      13      7

Average Turnaround Time = 6.75
Average Waiting Time = 2.50

Process returned 0 (0x0)   execution time : 9.643 s
Press any key to continue.
```

Algorithm	Average Waiting Time (ms)	Average Turnaround Time (ms)
FCFS (Without Arrival Time)	8.25	18.75
FCFS (With Arrival Time)	6.25	15.25
SJF (Non-Preemptive)	5.25	11.75
SJF (Preemptive)	4.00	9.50
Priority (Pre-emptive)	3.00	8.00
Round Robin (Quantum = 3)	6.75	14.50

